

GHG 微服務架構

所屬專案：dsp-ghg

最後更新：2026-03-02

1. 功能簡述

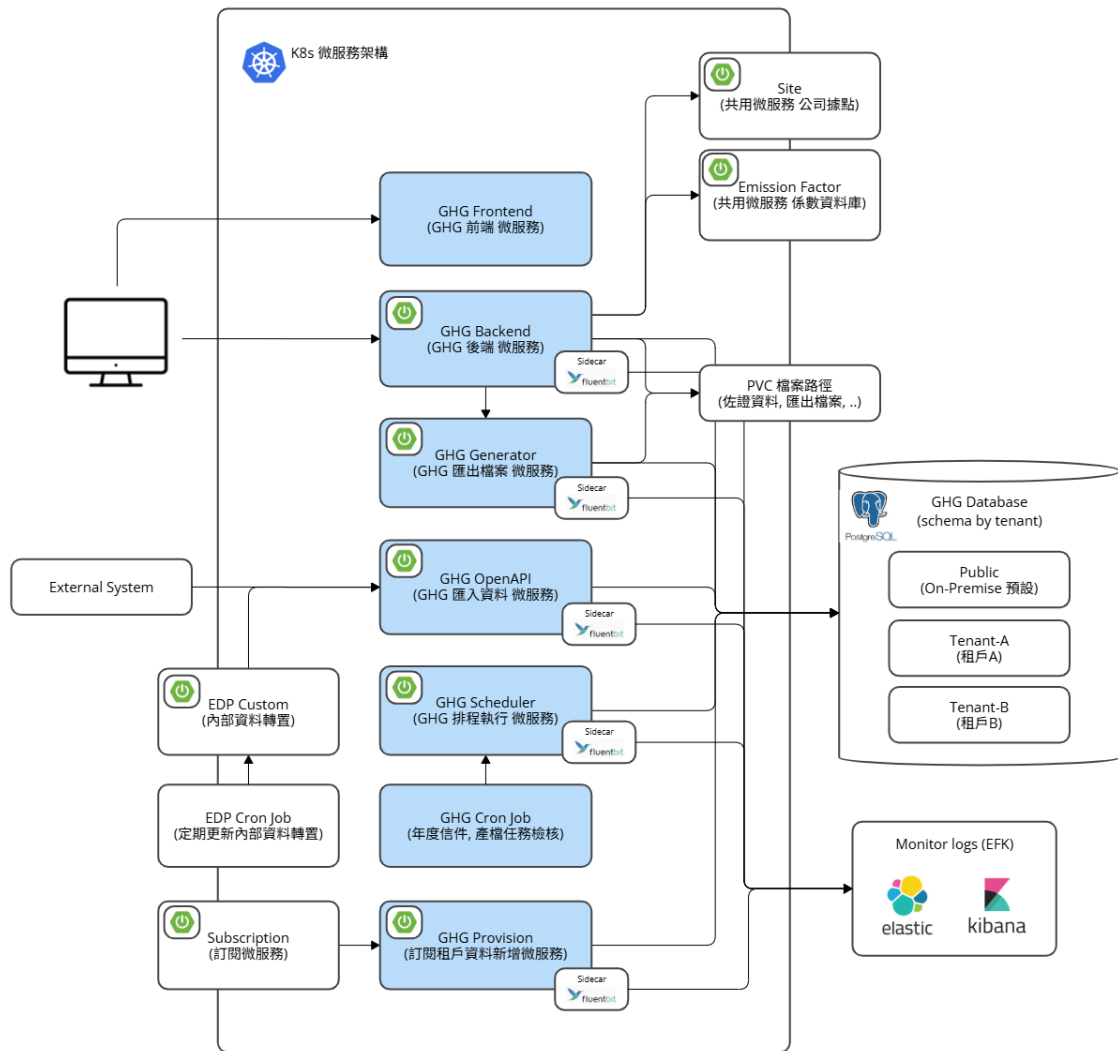
GHG 是 DSPBD 開發的平台，主要支援企業溫室氣體排放的盤查、計算、報告產出。

項目	技術
語言	Java 21
框架	Spring Boot 3.4.12
建構工具	Gradle 8.13
ORM	Spring Data JPA + Hibernate
資料庫	PostgreSQL (多租戶 Schema 隔離)
容器化	Docker + Helm + K8s

2. 架構設計

微服務架構

GHG 微服務架構

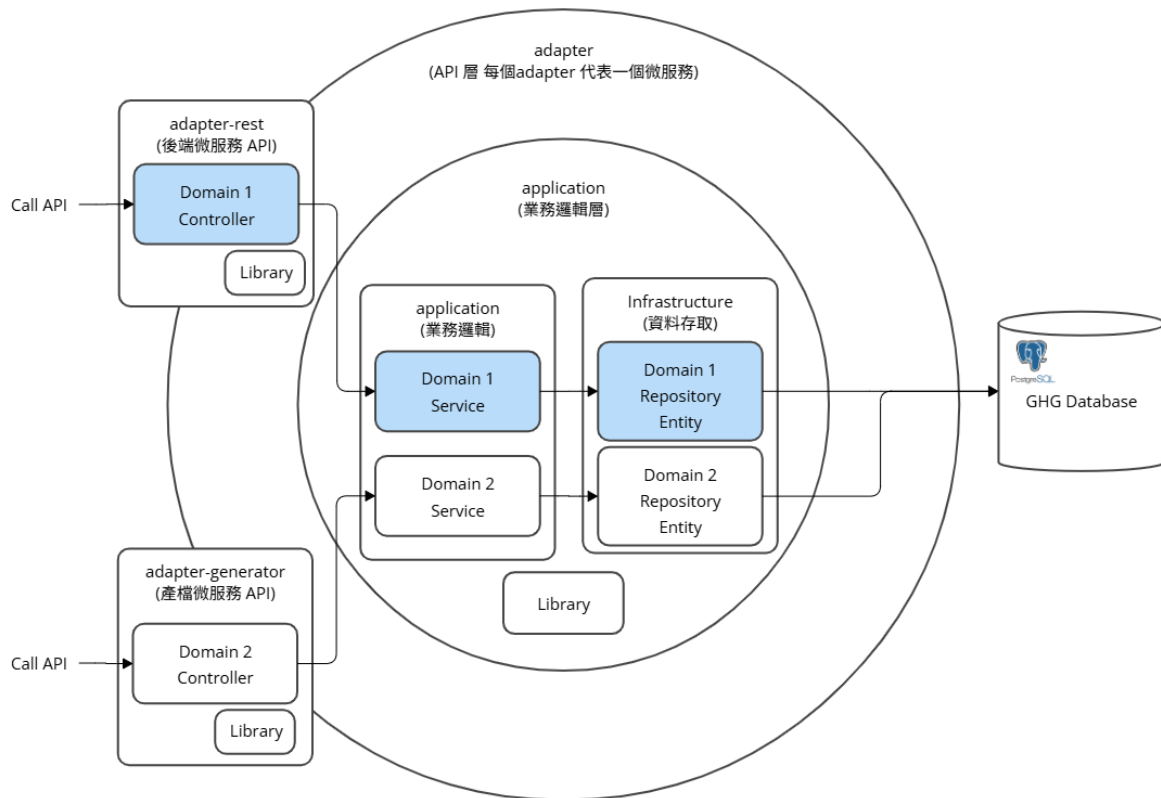


- GHG 微服務:

- GHG Frontend (GHG 前端 微服務)
- GHG Backend (GHG 後端 微服務)
- GHG Generator (GHG 匯出檔案 微服務)
- GHG OpenAPI (GHG 匯入資料 微服務)
- GHG Scheduler (GHG 排程執行 微服務)
- GHG CronJob (Multi Tenant Trigger, 年度信件, 產檔任務檢核)
- GHG Provision (訂閱租戶資料新增 微服務)

GHG 軟體架構

GHG 軟體架構



- adapter-*: Present Layer

e.g., rest, generator, openapi, scheduler, provision...

此Project包含程式啟動 Main Class

- application: Domain Layer

主要邏輯會在此Project實作

- common/

共用物件 (包含業務邏輯共用物件)

- application/

主要業務邏輯 DTO / Service / ...

- infrastructure/

Persistent Object

多模組結構

```
dsp-ghg/
├─ application/           # Domain Layer – 核心業務邏輯 ( 共享函式庫 )
├─ adapter-rest/          # 主要 API 微服務 ( GHG 平台入口 )
├─ adapter-generator/     # 產檔微服務 ( 清冊、報告書 )
├─ adapter-openapi/       # 外部資料匯入微服務 ( EDP 資料 )
├─ adapter-provision/     # 租戶佈建微服務 ( Schema 管理 )
├─ adapter-scheduler/     # 排程任務微服務
└─ charts/               # Helm Charts ( 管理 Dev, Test 環境的 K8s 設定檔 )
```

模組職責

模組	可獨立啟動	依賴 application	職責
adapter-rest	✓	✓	GHG 平台主 API 入口
adapter-generator	✓	✓	檔案匯出服務 (清冊 Excel、報告書 Word)
adapter-openapi	✓	✓	外部資料匯入接口 (EDP)
adapter-provision	✓	✗	租戶佈建服務 (Liquibase Schema Migration)
adapter-scheduler	✓	✓	排程任務執行
application	✗	—	核心業務邏輯

application 模組分層

```
application/src/main/java/com/deltaww/dsp/ghg/
├─ application/           # Service 層 – 業務邏輯 ( calculator, emissionsource, report... )
├─ common/               # 共用工具 ( config, constant, enums, i18n, utils ... )
└─ infrastructure/       # 持久層 – JPA Repository
```

DSP 共用Library

Library	用途
dsp-security	JWT 認證
dsp-rbac	角色權限控制
dsp-license	授權管理
dsp-liquibase	DB 版版控制

Library	用途
<code>dynamic-schema-inspector</code>	多租戶 Schema 動態切換
<code>file-storage-interface</code> / <code>local-storage-module</code>	檔案儲存

微服務間通訊 (OpenFeign)

- 連線基本設定在 FeignConfig
 - 包含 Timeout 時間, JWT Token, User Locale
 - 微服務之間往後傳遞需要的資訊

Client	用途
<code>IFileGeneratorClient</code>	adapter-rest → adapter-generator (觸發產檔)
<code>ISiteClient</code>	據點資料查詢
<code>ICompanyClient</code>	公司資料查詢
<code>IUserClient</code>	使用者資料查詢
<code>IAzureMapClient</code>	地圖資訊查詢

多租戶架構

- 每個租戶擁有獨立的 PostgreSQL Schema
- 請求層透過 JWT Token 中的 `tenantId` 識別租戶
- 實際 JPA 對 DB 進行操作時, `dynamic-schema-inspector` Library 會至換 SQL 的目標 DB Schema
- `adapter-provision` 負責新租戶的 Schema 初始化 & 更新各個租戶 DB Table 版本

5. 常見問題與排查

問題	排查方向
多租戶 Schema 切換失敗	檢查 JWT Token 中的 <code>tenantId</code> or Header <code>X-Tenant-Id</code> 設定, <code>dspTenantHolder.getCurrentTenant()</code> 可看出目前的 Tenant
微服務啟動失敗	檢查 <code>application.yaml</code> 環境變數 是否有新增到 ConfigMap, PostgreSQL 連線 Schema 是否存在, PostgreSQL 連線是否遇到 Permission Deny
OpenFeign 呼叫失敗	確認目標微服務是否啟動, URL 設定是否正確

6. 擴充指引

新增 CronJob 排程

使用時機在需要針對所有 Tenant 要統一資料轉移時，建議建立 CronJob，可手動觸發一次，因 CronJob 本身已經設計為 Multi-Tenant

1. 在 `application` 新增對應的業務邏輯
2. 在 `adapter-scheduler` 裡面的 `SchedulerController` 新增對應的 API
3. 新增 `charts/dsp-ghg-cronjob/templates` 對應的 cronjob template
4. 更新 `charts/dsp-ghg-cronjob/templates` 對應的 values yml 指定 API & 排程時間

新增微服務模組

1. 在專案根目錄建立 `adapter-xxx/` 模組
2. 在 `settings.gradle` 註冊模組
3. 設定 `build.gradle` 引用 `application` 模組
4. 建立對應的 Helm Chart 在 `charts/` 下
5. 更新 `Jenkinsfile` 加入 Image Build / Chart / Deploy 階段
6. 更新 Git Repo Jenkins CD Pipeline

新增 Domain 功能

1. 在 `application/` 中建立 Service + Repository
2. 在對應的 `adapter-xxx` 中建立 Controller (給前端用的功能 `adapter-rest`)
3. 設定 RBAC 權限 (`@RbacAuthorize`, `@RuleAuthorize`)

在 Local 啟用 多個微服務 (Rest & Generator)

詳見 [微服務啟用方法](#)